

PHOENIX: Resilient LLM Training with Hot-Swapping via Zero-Overhead Checkpoint

Haotian Xie*, Junlin Chen*, Mingkai Zheng, Lishan Yang, and Zhao Zhang

Rutgers University, George Mason University

{haotian.xie, junlin.chen110, mingkai.zheng, zhao.zhang}@rutgers.edu, lishan.yang@gmu.edu

Abstract—State-of-the-art large language model (LLM) training takes tens of thousands of graphics processing units (GPUs) for months and encounters failures across the software and hardware stack. Existing fault-tolerance mechanisms either impose non-trivial overhead during failure-free execution or suffer from prolonged recovery latency, particularly under scenarios where a small subset of compute nodes experience permanent failures. We present PHOENIX to simultaneously address both optimization objectives. PHOENIX incorporates a fault-tolerance mechanism that restores LLM training via hot-swapping, namely by replacing failed nodes with spare nodes without terminating the complete job. The hot-swapping of PHOENIX is enabled by two ideas: First, it exploits an off-critical-path in-memory checkpointing mechanism for spatial redundancy. Second, it introduces a communicator reconstruction protocol that replaces failed nodes with spare nodes at runtime. PHOENIX efficiently overlaps the in-memory checkpointing with computation, thus introducing zero overhead during error-free execution. Upon permanent node failures, PHOENIX can rebuild memory states with minimal recomputation by leveraging in-memory checkpoints. We evaluate PHOENIX across scales (up to 512 NVIDIA A100 GPUs) and LLMs (up to 65B parameters), and observe zero checkpoint overhead with hot-swapping recovery completing in under 40 seconds. These results show that PHOENIX simultaneously achieves both zero-overhead error-free execution and extremely low recovery cost.

Index Terms—fault tolerance, large language model training, distributed training, checkpointing, hot swapping, 3D parallelism

I. INTRODUCTION

LLM training runs on thousands to tens of thousands of GPUs, where node failures are no longer rare events but an expected operating condition [1], [2]. Published interruption statistics from a 32K-GPU LLM pre-training deployment report 678 unexpected interruptions, with GPU HBM memory faults, PCIe device failures, and NCCL watchdog timeouts alone accounting for 49.9% of all events [3]. The traditional checkpoint-restart solution periodically writes a persistent checkpoint, e.g., for every 1,000 steps [1], [4], to amortize the checkpointing overhead. Upon failures, the job terminates, and the user needs to start a new job that reloads the latest checkpoint. This approach pays for the recovery cost twice: It introduces backup overhead during normal execution, and when a failure occurs, it incurs both recovery latency and the recomputation cost for work performed after the latest checkpoint. On over-committed supercomputers, such as NERSC Perlmutter or TACC Vista, a double-bit memory error or an NCCL error can cause the complete job to quit. The user must

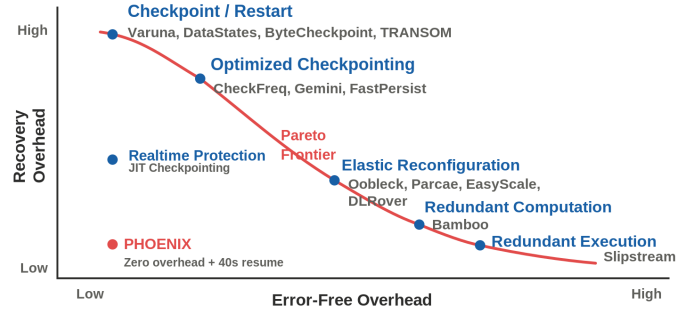


Fig. 1: Conceptual landscape of existing fault-tolerance strategies in the tradeoff space of error-free overhead and recovery cost

re-enter an already congested queue, incurring additional wait time.

Figure 1 conceptually visualizes existing fault-tolerant LLM training strategies in a tradeoff space between the error-free overhead and the recovery cost. Checkpointing strategies, such as Datastates-LLM [5], ByteCheckpoint [6], TRANSOM [7], and Universal Checkpointing [8], substantially improve checkpoint throughput and flexibility. However, they still recover by restarting training from a persistent checkpoint, which may lead to a non-trivial cost due to recomputation for the lost training progress. Prior in-memory checkpointing strategies [9] reduce write latency, but assume the in-memory checkpoint is accessible in the presence of a failure, which is not a valid assumption if the failure is permanent. On the other hand, elastic fault-tolerance strategies, such as Bamboo [10], tolerate membership changes by exploiting bubbles in pipeline-parallelism. But these approaches are with degraded training throughput.

Our goal in this work is to reduce the overhead for both error-free execution and recovery. For error-free execution, the LLM framework needs to persist checkpoints asynchronously to avoid potential contention with the training processes. To lower recovery cost, the LLM framework needs to checkpoint at every iteration to minimize recomputation in the presence of a failure. Further, the LLM framework needs to avoid a full restart, which introduces additional overhead for global job termination, checkpoint consistency verification, filesystem I/O, and memory state reconstruction. These challenging requirements motivate the design of PHOENIX, a fault-tolerant system for LLM training via hot-swapping that leverages zero-overhead asynchronous in-memory checkpointing in complex

parallelism.

PHOENIX is designed with two unique insights. The first insight is that the memory states during LLM training are distributed using different strategies in complex parallelism, e.g., 3D parallelism. A typical setting of 3D parallelism, which overlays sharded data-parallelism with pipeline-parallelism and tensor-parallelism, replicates the model shards across data-parallel peers while distributing the optimizer states (i.e., the first and second moments) across GPUs within a data-parallel rank (i.e., a group of GPUs). Upon a permanent node failure, model parameters remain reconstructible from healthy ranks that share the same logical partition, but the optimizer state shards cannot be recovered. Thus, we design a low-overhead, across-nodes in-memory optimizer state checkpoint strategy that effectively overlaps communication with computation, so that PHOENIX can replicate optimizer state shards in every iteration while avoiding exposed checkpoint overhead. A second insight is that PHOENIX views node failure recovery as an online topology-repair problem rather than a full-job restart. Thus, we design a distributed communicator reconstruction protocol to replace failed nodes with spare nodes to avoid full allocation restart.

We implement PHOENIX on top of PyTorch [11] and Megatron-LM [4], but the design relies only on generic capabilities: sharded model and optimizer states, host-side spare-node management, shared recovery coordination, and runtime group reconstruction. PHOENIX includes an error-classification layer that distinguishes transient and permanent failures, allowing PHOENIX to apply topology-changing recovery for failures that actually invalidate node membership. We evaluate PHOENIX on two production HPC systems, i.e., NERSC Perlmutter and TACC Vista, using up to 512 A100 GPUs and 64 GH200 GPUs, respectively. We inject node failures into live training jobs with reserved spare nodes, which is a standard methodology in research on fault tolerance and recovery [12]. Across both platforms, we observe negligible per-step checkpointing overhead on the training critical path. Moreover, PHOENIX consistently completes node replacement and restores training around 40 s after a failure.

Our main contributions are as follows:

- We present PHOENIX, an online hot-swapping recovery mechanism for large-scale LLM training.
- We design an asynchronous neighbor-memory replication strategy for optimizer state shards.
- We evaluate PHOENIX on up to 512 GPUs and 65B model size, showing zero-overhead during error-free execution and a consistent recovery cost within 40 s.

The zero-overhead checkpointing of PHOENIX depends on the GPU cluster configuration, the model size, and the model deployment strategies. Although PHOENIX is only examined on Perlmutter and Vista, we expect PHOENIX to be effective across many other GPU-dense supercomputers. This is because the checkpointing overhead is one order of magnitude lower than computation, as shown in Figure 6, on the current generation machine. Given the trend of GPU compute capability and interconnect bandwidth increase on the coming NVIDIA

Grace-Blackwell supercomputers, PHOENIX is expected to continue achieving zero-overhead checkpointing.

II. BACKGROUND

A. Large Language Model Training

LLM capabilities often increase with model and dataset size [13], [14]. Thus, to overcome training time and memory limitations, researchers have designed various distributed training strategies, including data-, tensor-, pipeline-, and sequence-parallelism [15].

Data parallelism replicates a model across processors. Training then iterates five steps: 1) I/O, 2) forward compute, 3) gradient evaluation, 4) gradient exchange, and 5) variable update. **3D-parallelism** partitions a model across processors by layering data-parallelism over model- and pipeline-parallelism. Processors are partitioned into groups (*Data Parallel Rank*) and each group hosts a complete copy of the model. Inside each group, the model is spread across nodes by layers to form the pipeline stages. Inside each node, layers (tensors) are evenly distributed across multiple processors in what is referred to as *model* or *tensor* parallelism. With the data-parallel approach, the only communications are the broadcast at the start of training and the gradient exchange in each iteration. In contrast, 3D-parallelism requires intra- and inter-node communication for both forward and backward computation. **Long sequence support**, a fourth dimension in LLM training, enables learning from larger contexts. Researchers have designed sequence-parallelism with various partitioning and communication strategies over the query, key, and value matrices [16]–[18]. Among existing methods, ring attention achieves the lowest memory overhead by using an outer loop of query blocks (across GPUs) and inner loops for key-value blocks (within a GPU). These distributed LLM training strategies complicate the design of fault-tolerant LLM training systems, as they are prone to suboptimal recovery decisions and unnecessary overhead in the error-free phase.

B. Errors in Systems

In computer systems, hardware transient faults (i.e., soft errors) [19], [20] can be triggered by high energy particles from cosmic radiation [21], shrinking transistors, and low voltage operation [22]. These hardware faults manifest as bit-flips and can range from a single bit to multiple bits and even to hundreds of lines of memory cells [20]. We explain the concepts of fault/error/failure [23] next: A hardware *fault* propagates in the system; once it reaches the software level, it becomes an *error* that is visible to the program. That error further propagates in the program execution, which can lead to three possible outcomes [24]: 1) no effect (i.e., correct output), 2) silent data corruption, 3) *failures* including application crashes and unresponsive systems. The first two types of outcome are not noticeable to users, although as quantified in prior research [25], [26], silent data corruption can alter the model parameters produced by training. For noticeable errors, scientists have designed a comprehensive set of technologies for detecting and tolerating the errors. As our choices of

recovery methods will vary greatly with error type, error characterization and analysis are important for understanding and, more importantly, for detecting and correcting errors efficiently [27]. Accurate understanding of errors, and, if possible, accurate prediction of imminent errors and failures, are important if we are to perform timely mitigation actions that avoid data loss and increases system dependability [27].

III. RELATED WORKS

Existing fault tolerance solutions can be categorized into two approaches. *Checkpointing* methods recover model states from recent checkpoints, while *Exclude-and-Run* methods exclude faulty devices while allowing healthy workers to continue without interruption.

Early neural network training approaches employed elastic training methods to handle faults, with frameworks like Horovod [28] and PyTorch [11] offering APIs for failure recovery. However, these methods primarily address data parallelism for models that can fit on a single GPU. As the size of models grows, so has the likelihood of encountering failures during training [2], [29], [30]. EasyScale [31] and DLRouter [32] take hybrid parallelism into account, then propose resilient training pipelines that can be applied to thousands of GPUs. Failure rates are even higher when using spot training [10], [33] in clusters. Varuna [33] uses periodic checkpoints to recover from one or more failures, while Bamboo [10] takes advantage of pipeline parallelism by utilizing bubbles between neighboring stages to create redundancy. However, both methods introduce additional re-computing or checkpointing overhead, which can significantly impact training efficiency as failure rates increase.

Oobleck [34] achieves a balance between the throughput and fault tolerance by introducing pipeline templates that enable rapid failure recovery and full GPU utilization. Parcae [35] leverages a novel metric, liveput, to dynamically reconfigure the parallelization strategy in preemptive environments, maximizing throughput. Since frequent adjustments to hybrid parallelism can lead to overhead, Slipstream [36] exploits healthy GPUs to take over computations from failed GPUs in a pipeline stage without changing the strategy.

Due to the high overhead associated with checkpoint-based recovery, Checkfreq [37] introduces a fine-grained, automated checkpointing mechanism that overlaps computation with saving model states, optimizing checkpoint frequency through runtime profiling. Gemini [38] leverages hierarchical memory to create a more efficient checkpointing system, while FastPersist [39], developed by DeepSpeed, optimizes checkpoint writing from NVMe to SSDs in parallel, significantly improving checkpointing efficiency. In fault-tolerant scenarios, Just-in-time checkpoint [40] uses data-parallel replicas in 3D parallelism and leverages the iterative nature of DNN training to implement a mechanism that allows at most one step of re-computation in case of failure. Techniques [41]–[43] that consider model-specific characteristics (e.g., recommendation models that do not update all parameters simultaneously)

further enhance fault recovery efficiency. For example, CheckN-Run [42] copies only modified parameters to host memory, and CPR [43] recovers only the failed GPU using checkpoints while allowing other GPUs to continue working.

In general, checkpoint-based methods ensure that the model returns to a globally stable state after a failure, but may introduce additional synchronization overhead that lowers training efficiency, particularly when failure rates are high. Exclude-and-run methods avoid this overhead by dynamically adjusting the parallelization strategy or redistributing computations to functioning workers, but the imbalanced distribution of a mini-batch across the reduced number of GPUs may lead to straggling processes. Existing in-memory approaches such as JIT checkpointing can avoid checkpoint-interval replay, but their reliance on failure-time state reconstruction through function overloading can introduce nested recovery logic and unstable behavior under complex failure scenarios [44]. As shown in Table I, PHOENIX differs fundamentally from all three categories: rather than constructing checkpoints at failure time, relying on redundant computation, or reducing the worker set. PHOENIX maintains a continuously recoverable state in distributed memory and performs recovery through online node replacement and communication reconfiguration, eliminating full-job restart and reducing recovery to a bounded topology repair process.

IV. DISRUPTION COST MODELING

We model the disruption caused by failures as the sum of three terms:

- (1) The overhead paid during failure-free execution to maintain recoverability.
- (2) The latency required to restore execution after a failure.
- (3) The amount of work that must be replayed after recovery.

For periodic checkpoint-restart, if checkpoints are taken every K steps and each step takes t_{step} , then the replay cost of a failure is $R \cdot t_{\text{step}}$, where R is the number of steps executed since the last checkpoint. Under a uniform-failure approximation, $\mathbb{E}[R] \approx (K - 1)/2$. However, failures in large-scale training are not necessarily uniform; they may be bursty and system-dependent. We therefore also consider an empirical replay distribution derived from failure traces, replacing the uniform approximation with $\mathbb{E}[R] = \sum_r r \Pr(R = r)$.

This yields the expected disruption per failure under checkpoint-restart:

$$\mathbb{E}[C_{\text{ckpt}}] = C_{\text{overhead}}^{\text{ckpt}} + \mathbb{E}[T_{\text{restart}}] + t_{\text{step}} \cdot \mathbb{E}[R].$$

We treat replay as wasted work relative to failure-free execution, even though it is re-executed as part of resumed training.

To relate per-failure disruption to overall training efficiency, we incorporate the system-level failure rate using the mean time to failure (MTTF). Let MTTF_{sys} denote the average time between failures at the system scale, and $\lambda = 1/\text{MTTF}_{\text{sys}}$ the corresponding failure rate. The expected disruption per unit time can then be approximated as:

TABLE I: Comparison of failure recovery approaches. PHOENIX maintains continuously recoverable state and enables online recovery via topology repair.

Method	Checkpoint Timing	Recovery Mechanism	Replay
Checkpoint-Restart	Periodic	Restart + reload	Yes (interval)
JIT Checkpointing	On failure	Restart + reconstruct state	≤ 1 minibatch
Bamboo	None (redundant compute)	Continue from redundant work	None
PHOENIX	Per-step (in-memory)	Node replacement + reconfiguration	≤ 1 step

$$\mathbb{E}[C_{\text{rate}}] = \lambda \cdot \mathbb{E}[C_{\text{ckpt}}].$$

We further approximate training as alternating between failure-free execution intervals of length MTTF_{sys} and disruption intervals of expected duration $\mathbb{E}[C_{\text{ckpt}}]$, yielding the effective training efficiency:

$$\eta \approx \frac{\text{MTTF}_{\text{sys}}}{\text{MTTF}_{\text{sys}} + \mathbb{E}[C_{\text{ckpt}}]}.$$

As system scale increases, MTTF_{sys} typically decreases due to the growing number of failure-prone components, amplifying the impact of recovery cost.

FT-HSDP reports that at $O(100K)$ GPUs, failures occur roughly once every 18 minutes, synchronous recovery stalls the entire job for about 10 minutes, and persistent checkpoints are commonly written every 100 steps, with a typical step time of about 20 seconds. Under the uniform model, this corresponds to an expected replay cost of approximately $(K/2) \cdot t_{\text{step}} \approx 16.7$ minutes, which is comparable to or even exceeds the restart stall itself. This highlights that, at a large scale, replay and restart lead to low overall training efficiency.

PHOENIX reduces disruption per failure by eliminating checkpoint-interval replay and replacing full-job restart with fast hot-swapping recovery. The resulting disruption can be expressed as:

$$\mathbb{E}[C_{\text{PHOENIX}}] = C_{\text{overhead}}^{\text{PHOENIX}} + \mathbb{E}[T_{\text{hotswap}}] + t_{\text{step}} \cdot \mathbb{E}[\phi],$$

where $\phi \in [0,1)$ denotes the fractional progress of the interrupted step. This formulation captures that PHOENIX reduces replay to at most the interrupted step and avoids full-job restart, significantly lowering the disruption per failure.

V. SYSTEM DESIGN

In this section, we describe how PHOENIX overlaps state protection with training execution and ensures fast, correct recovery without interrupting the global training workflow.

A. Overview

As shown in Section IV, failure-induced disruption in large-scale training is dominated by restart latency and replay cost under checkpoint-restart. This motivates a design that eliminates checkpoint-interval replay and avoids full-job restart. PHOENIX achieves this by transforming recovery into an online topology repair problem: instead of restoring execution from a persistent checkpoint, PHOENIX maintains a continuously recoverable state in distributed memory and replaces

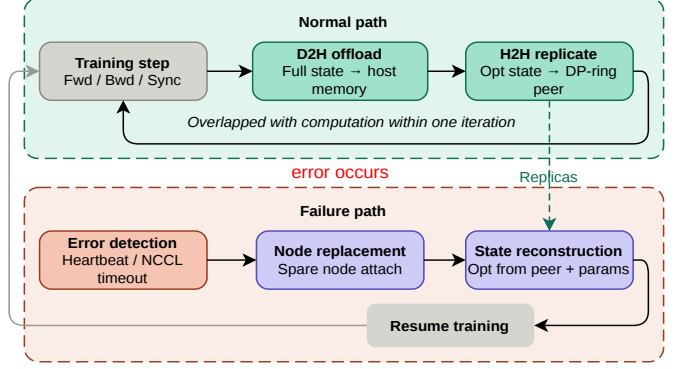


Fig. 2: System overview of PHOENIX. The normal path (top) runs a two-phase offload pipeline overlapped with each training step. The failure path (bottom) detects faults, replaces the failed node, and reconstructs state from in-memory replicas.

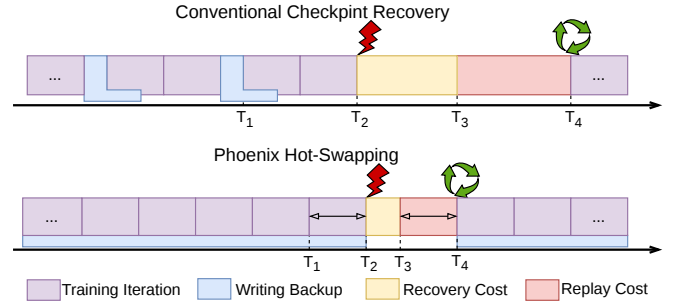


Fig. 3: Timeline comparison of conventional checkpoint recovery and PHOENIX. PHOENIX overlaps protection with training and reduces disruption after failure.

failed nodes with healthy ones, allowing training to resume without restarting the job.

As illustrated in Figure 2, PHOENIX comprises three components that separate normal-path state protection from failure-path recovery:

- **Asynchronous Offload Pipeline** (Section V-B) runs on the normal path once per training iteration. It copies the full rank state to host memory and replicates optimizer-state shards to a DP-ring peer. Both phases are overlapped with forward, backward, and gradient synchronization, keeping redundancy cost off the critical path.
- **Error Detection and Isolation** (Section VI-B) monitors node health via heartbeats and NCCL timeouts. When a failure is detected, it traps the exception before it propagates to the job manager, holding surviving ranks

at a barrier while a replacement node is provisioned.

- **Recovery Policy** (Section V-C) executes on the failure path. It attaches a spare node, rebuilds communication groups, and reconstructs the missing state from in-memory replicas. Training resumes from the last completed step with at most one step of re-execution.

During normal execution, only the offload pipeline is active, and its cost is hidden within each training iteration. When a failure occurs, the error detection module triggers the recovery policy, which leverages the remote replicas to restore training without a full job restart.

Figure 3 compares conventional checkpoint-restart with PHOENIX from a timeline perspective. Let T_1 denote the latest recoverable state before failure. T_2 is the failure time. T_3 is the point at which training resumes, and T_4 is the point at which execution catches up to a failure-free trajectory. In conventional checkpoint-restart, periodic backup operations are placed on the training critical path, and a failure introduces two sources of disruption: explicit recovery latency and replay of lost work between T_1 and T_2 . As a result, training cannot resume until the system reloads state and reconstructs execution, and additional time is spent re-executing previously completed iterations before reaching T_4 .

In contrast, PHOENIX decouples state protection from the critical path by continuously maintaining recoverable state in memory and overlapping redundancy operations with normal execution. Upon failure, the system performs online node replacement and reconstructs only the missing state, allowing training to resume near T_3 with at most one step of re-execution. This design significantly reduces both replay cost and end-to-end recovery latency, effectively shrinking the disruption window between failure and full recovery.

B. Asynchronous Offload Pipeline

The offload pipeline runs once per training iteration, overlapped with the forward pass, backward pass, and gradient synchronization of the same iteration as illustrated in Figure 4. It proceeds in two phases whose payloads differ by design.

a) *Device-to-host (D2H)*: The full rank state, which comprises the optimizer-state shard, the local model-parameter shard, and a metadata envelope (step index, parallelism configuration, RNG states), is packed into a contiguous buffer and copied asynchronously to pinned host memory on a dedicated CUDA stream. Retaining the complete rank state on the host enables fast local recovery (e.g., after a transient GPU error) without any network transfer.

b) *Host-to-host (H2H)*: Only the optimizer-state shard and the metadata envelope are transmitted to the peer node in a ring topology, which is the rank sharing the same tensor-parallel and pipeline-parallel positions but with a different data-parallel index. Model parameters are excluded: upon node failure, the replacement rank can reconstruct its parameter shard from healthy ranks at the same TP/PP coordinate via a targeted collective, so replicating parameters to a peer in advance would consume network bandwidth without reducing recovery latency. A background worker thread waits on the

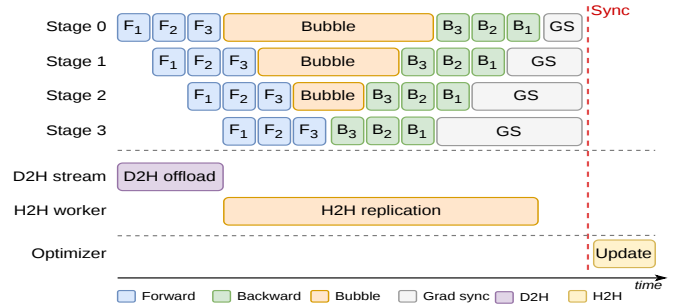


Fig. 4: Illustration of the asynchronous offload pipeline across pipeline-parallel stages. Computation (forward and backward passes) is interleaved with gradient synchronization, while device-to-host (D2H) offload and host-to-host (H2H) replication progress concurrently on separate execution paths. The timeline highlights the overlap between compute and communication throughout an iteration.

D2H completion event and then sends the optimizer payload to the ring peer via chunked MPI over the DP-local communicator.

This payload is asymmetric, full rank state for D2H versus optimizer-only for H2H, because of the bandwidth difference of the system. Local PCIe transfers are fast, whereas inter-node network bandwidth is more constrained. By transmitting only the optimizer shard over the network, the H2H phase reduces its transfer volume, increasing the likelihood that both phases complete within a single training iteration.

C. Recovery Policy and Correctness Guarantees

PHOENIX handles failures by determining whether the affected node can continue participating in distributed execution.

Failure Model. We consider two classes of failures. (1) *Transient failures*, where the affected node remains operational and its state remains valid. (2) *Permanent failures*, where a node or its state is permanently lost and can no longer contribute its assigned shards. In the latter case, the system replaces the failed node with a spare node and reconstructs the missing state.

Failures that may compromise training semantics, such as software bugs, numerical errors, or silent data corruption, are treated as outside the scope of PHOENIX and handled by an external fallback mechanism.

System Invariants. To ensure correctness under failures, PHOENIX maintains the following invariants:

- **I1 (Topology Consistency).** All involved nodes agree on a single communication topology and shard placement.
- **I2 (Shard Completeness).** For every logical shard, there exists exactly one valid owner among the involved nodes.
- **I3 (Optimizer Availability).** The optimizer state for every shard is available from in-memory replicas.
- **I4 (Step Atomicity).** Training progresses in discrete steps; partial progress from a failed step is not committed.

During transient failures, the LLM training system preserves all invariants. These failures can be recovered in place without modifying involved nodes or the communication topology.

Recovery Procedure. For permanent failures, PHOENIX recovers in three stages. First, the system selects a spare node and updates the set of involved nodes. This change invalidates the pre-failure communication topology. Second, all involved nodes are quiesced at a safe boundary, and the system re-establishes a consistent communication topology and shard placement across all nodes. This step ensures that Invariants I1 and I2 hold before execution resumes. Third, the system reconstructs the missing state on the spare node. Model parameters are reconstructed from healthy nodes that share the same partition, while optimizer state is restored from in-memory replicas. This step ensures that Invariant I3 holds. Finally, execution resumes from the last completed step, ensuring that Invariant I4 is preserved.

a) *Tolerating Multi-Node Failures via Multi-Peer Replication.* By default, PHOENIX replicates each optimizer-state shard to one DP-ring neighbor, tolerating any single-node failure. To guard against concurrent multi-node failures, PHOENIX can be configured to replicate each shard to k distinct DP peers, with the additional H2H cost scaling linearly in k but remaining overlappable with computation for a sufficiently small k .

A shard is irrecoverably lost only if its owner and all k replica holders fail within the same step. We quantify this risk using published failure statistics from Meta’s 100K-GPU deployment, which reports one interruption roughly every 18 minutes [45]. With a step time of approximately 20s, this corresponds to a per-node per-step failure probability on the order of $p \approx 10^{-6}$, which is an aggressive estimate that we adopt as an upper bound for our analysis.

Independent failures. When replica placement is topology-unaware, we assume node failures are independent. The probability that a specific shard loses all $k+1$ copies in a single step is p^{k+1} . Applying a union bound over D data-parallel shards, the per-step probability of any shard becoming unrecoverable is:

$$P_{\text{loss}} \leq D \cdot p^{k+1}. \quad (1)$$

With $D = 128$ and $k = 1$, this yields $P_{\text{loss}} \leq 1.28 \times 10^{-10}$ per step, or a cumulative probability of 1.28×10^{-5} over a 10^5 -step run—fewer than one event per 100 full training runs. Setting $k = 2$ reduces the cumulative probability to 1.28×10^{-11} , which is effectively zero.

Correlated failures with topology-aware placement. The independent-failure analysis breaks down when a shared infrastructure component—such as a top-of-rack (ToR) switch, a network cable, or a power distribution unit—fails and takes down multiple co-located nodes simultaneously. If a shard’s owner and its replica reside under the same switch, a single switch failure can destroy both copies.

To mitigate this, PHOENIX can leverage cluster topology information to place the k replicas on nodes under k distinct switches or racks, ensuring that a single infrastructure event affects at most one copy of any given shard. Let q denote the per-domain per-step failure probability (e.g., a ToR switch failure rate). With replicas distributed across independent

failure domains, the per-shard loss probability becomes q^{k+1} . Even under a relatively high domain failure rate of $q = 10^{-4}$, $k = 1$ yields a cumulative risk of $D \cdot q^2 \times 10^5 = 0.128$ over 10^5 steps—non-negligible for safety-critical runs. Setting $k = 2$ reduces this to $D \cdot q^3 \times 10^5 = 1.28 \times 10^{-5}$, rendering correlated shard loss negligible.

In practice, $k = 1$ with topology-aware placement suffices for the vast majority of production scenarios, while $k = 2$ provides an additional safety margin for deployments with frequent infrastructure-level failures.

VI. IMPLEMENTATION

We implement PHOENIX as a hybrid design that combines a data-plane extension to Megatron-LM with an external control plane. The data plane integrates directly with Megatron-LM’s distributed execution model, including checkpoint meta-data, sharded optimizer abstractions, and 3D-parallel process-group organization, leaving the failure-free execution path unchanged when the mechanism is disabled.

The control plane is implemented as a lightweight standalone service that coordinates failure detection, node replacement, and recovery orchestration across all involved nodes. It provides a persistent coordination layer independent of the training processes, enabling recovery to proceed even when a subset of training nodes fail.

The implementation spans four key components:

- (1) the program entry path for initializing failure-aware execution and registering with the control plane,
- (2) the training-step scheduler for enforcing safe quiescence and recovery boundaries,
- (3) the checkpoint and optimizer wrapper for maintaining in-memory recoverable state, and
- (4) an asynchronous MPI-based transport service for state replication and recovery.

We implement PHOENIX as an opt-in extension inside Megatron-LM. The in-process design has two advantages over an external wrapper: it reuses Megatron’s distributed-optimizer serialization and 3D-parallel process-group layout directly, avoiding a redundant checkpoint stack; and when the mechanism is disabled, no interception layer is present in the training loop, so the failure-free path incurs zero overhead.

A. In-Memory State Protection

This component maintains a consistent, immediately recoverable snapshot of each rank’s training state in host memory at all times. As shown in Figure 5 offload manager uses a ping-pong scheme for the local snapshot buffer: two host-side buffers alternate roles across iterations, one holding the committed snapshot from iteration $k-1$ and the other serving as the staging target for iteration k . If a failure interrupts the current offload or the subsequent peer transfer, the in-progress buffer may be incomplete, but the previous buffer remains intact and immediately usable for recovery. This design guarantees that a valid recovery point is always present in host memory without requiring the offload to be atomic.

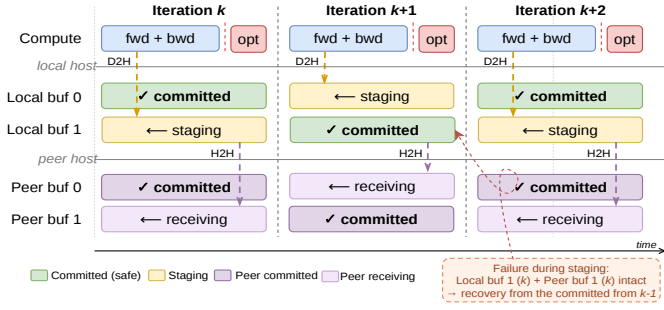


Fig. 5: Ping-pong double buffering across iterations. Two local buffers alternate between committed and staging states, while peer buffers alternate between committed and receiving. Computation overlaps with data movement (D2H and H2H), and buffer roles switch every iteration.

TABLE II: Error classification and recovery actions in PHOENIX.

Failure Category	Action
Transient communication faults	Local recovery first
GPU memory faults	Node replacement
PCIe / host / kernel / reboot faults	Node replacement
Network or storage faults	Node replacement
Software bugs / Unknown failures	External fallback

The buffer received from the ring predecessor is maintained separately and follows the same alternation logic.

The training loop exposes a single synchronization point: a wait immediately before the optimizer step that guarantees the peer-resident replica is fully committed before the optimizer mutates the protected state. In the common case, both D2H and H2H transfers complete within the iteration’s compute window, so this wait observes an already-finished offload and adds no exposed stall.

B. Error Classification

Following the failure model defined in Section V-C, PHOENIX implements a classification layer that maps observed failures to recovery actions as illustrated in Table II. The classification is driven by runtime signals that indicate loss of node participation or loss of reliable state. Device-level failures (e.g., GPU memory errors), system-level failures (e.g., host crashes or reboots), and persistent communication failures are treated as node-replacement events, as they prevent the node from safely contributing its assigned shards. Transient communication failures are initially handled as locally recoverable when the node remains responsive, but are elevated if they lead to sustained communication failure or node unavailability. When a failure is classified as locally recoverable, PHOENIX performs a local reset by clearing the node’s state and reinitializing it as a replacement node. From the system perspective, this is equivalent to a special case of node replacement where the failed node re-enters the system after reset. For clarity, we treat both cases uniformly as hot-swapping, and focus the following discussion on node replacement.

To evaluate the system under controlled yet realistic failure conditions, we inject errors during live training runs using timer-based triggers. At predefined execution points, the timer activates different fault types, including fail-stop events (via process termination), NCCL communication errors and storage-related errors. This unified injection mechanism enables reproducible evaluation of both abrupt node failures and runtime error conditions within the same framework.

C. Control Plane

This component provides a fault-isolated coordination substrate for involved nodes, maintaining globally consistent recovery metadata throughout failure handling and node replacement. PHOENIX implements the control plane as a standalone TCPStore-based service running as an independent process. It maintains a key-value store to recover metadata, including failure notifications, recovery epochs, node assignments, and synchronization signals. Because the control plane is decoupled from training processes, it remains available when a subset of nodes fails, allowing surviving and replacement nodes to continue coordination.

The control plane supports dynamic node participation, allowing nodes to join during recovery without requiring global re-initialization. Each involved node publishes a recovery descriptor that encodes its logical role, including shard ownership and parallel partition placement. These descriptors are used to derive deterministic mappings between source and replacement nodes based on logical shard identity.

D. Recovery Scheduling and Topology Reconfiguration

This component orchestrates the recovery process across involved nodes, ensuring consistent execution progression during node replacement and topology reconstruction. When a failure requires node replacement, the runtime initiates a recovery epoch and coordinates all involved nodes through the control plane, satisfying I1 mentioned in V-C. Each node captures its local recovery state in memory and synchronizes before recovery. Nodes are paused at a step boundary to establish a consistent execution point.

A spare node is activated and incorporated into the set of involved nodes. The runtime reconstructs communication groups and shard placement based on the updated topology, and all nodes resume execution under the same communication configuration. Recovery epochs are serialized, so that a new recovery begins only after the current one completes.

E. State Restoration and Transport

After topology reconfiguration, PHOENIX reconstructs the state of failed shards on the spare node. Model parameters are restored by transferring shards from source nodes that share the same logical partition, while optimizer state is restored directly from in-memory replicas maintained during training, avoiding access to persistent checkpoints.

To ensure correct reconstruction under dynamic rank reassignment, source and replacement nodes are matched using logical shard identity derived from recovery descriptors rather

than post-failure rank indices. This guarantees that each shard is reconstructed from the correct source regardless of how ranks are reassigned, corresponding to I2 and I3. State transfer is implemented by an asynchronous MPI-based transport layer, which allows communication to overlap with recovery preparation on the receiving node and reduces recovery latency.

F. Training Resume

Once model parameters and optimizer state are restored, the spare node assumes the role of the failed node and joins the set of involved nodes, ensuring the I4 completeness. Training resumes under the reconstructed communication topology from the last completed step, and any partial work from the interrupted step is discarded to preserve consistency. If a subsequent failure occurs, PHOENIX initiates a new recovery epoch following the same procedure.

VII. EXPERIMENTAL EVALUATION

We evaluate PHOENIX along three dimensions. First, we quantify its checkpoint overhead in the error-free case. Second, we measure the recovery cost of conventional checkpoint/restart under failures. Third, using an error model, we compare the end-to-end time-to-solution of PHOENIX against checkpoint-based recovery under realistic failure scenarios. Across the configurations we study, PHOENIX consistently reduces total wall-clock time.

A. Experimental Setup

1) *Platforms*: We evaluate PHOENIX on two production supercomputers: Perlmutter at NERSC and Vista at TACC.

Perlmutter is an HPE Cray EX system. We use its GPU nodes, with each containing one 64-core AMD EPYC 7763 CPU, four NVIDIA A100 GPUs, and 256 GB of DDR4 host memory. The four GPUs are interconnected via third-generation NVLink. Each node is equipped with four HPE Slingshot 11 NICs, each providing 200 Gb/s, for an aggregated bandwidth of 800 Gb/s per node. GPU-CPU and NIC-CPU connectivity are both provided through PCIe 4.0. The Perlmutter interconnect uses a three-hop dragonfly topology.

Vista is an NVIDIA Grace-based supercomputer at TACC. We use Grace-Hopper (GH) nodes, each containing one 72-core NVIDIA Grace CPU and one NVIDIA H200 GPU with 96 GB of HBM3 memory, connected through the Grace-Hopper superchip interconnect. Each node also provides 116 GB of LPDDR host memory and connects to the system fabric via NVIDIA NDR InfiniBand at 400 Gb/s. Vista employs a fat-tree topology.

2) *Training Configuration*: We train GPT-style transformer models from 0.6 B to 65 B parameters, varying the memory footprint by two orders of magnitude.

Unless otherwise noted, all experiments use full-precision (FP32) training: model parameters, main gradients, and Adam optimizer states (first and second moments, together with the FP32 master copy) are stored in `torch.float32`, with BF16 and FP16 disabled. This setting maximizes the per-rank state volume and therefore represents a demanding configuration for PHOENIX’s offload path.

We use Megatron-LM with 3D parallelism and ZeRO-2 distributed optimizer. Unless otherwise noted, all experiments use the 1F1B pipeline schedule. We evaluate PHOENIX across multiple parallel configurations from 8 to 512 GPUs.

B. PHOENIX’s performance

We design our evaluation to answer two key questions about PHOENIX:

- (1) Does per-iteration in-memory checkpointing introduce any runtime overhead under scaling?
- (2) Does hot-swapping recovery remain efficient as system scale increases?

To this end, we conduct two sets of experiments. Firstly, we evaluate the runtime overhead of PHOENIX under weak scaling, examining whether enabling per-iteration checkpointing affects training step time as model size and cluster size grow. Secondly, we measure the recovery latency of hot-swapping across different cluster scales to characterize its scalability and identify dominant cost components.

1) *Per-iteration Checkpoint Overhead under Scaling*: We first evaluate whether PHOENIX introduces any checkpoint overhead in the error-free case. Figure 6 shows weak scaling for the 2.3B GPT model under a fixed parallel topology, with TP=4 and PP=4 held constant. As the system scales from 32 to 256 GPUs, we increase the global batch size proportionally to the data-parallel degree, keeping the per-rank workload unchanged. To report per-iteration measurement, we discard the first 10 iterations as warm-up, remove the slowest five percent of the remaining iterations, and average the rest. Across all scales, PHOENIX closely tracks the baseline. On eight nodes, the baseline averages 2442.13 ms while PHOENIX averages 2470.15 ms. On 16 nodes, the two are nearly identical (2560.35 ms vs. 2561.75 ms). On 32 and 64 nodes, PHOENIX is marginally faster (2696.62 ms vs. 2701.87 ms and 3122.86 ms vs. 3134.81 ms, respectively). The differences are small and non-monotonic, with no trend of increasing overhead as the system grows, indicating that PHOENIX introduces negligible runtime overhead during error-free execution.

In addition to the fixed-topology weak-scaling results above, we also evaluate PHOENIX across a range of 3D parallel layouts on 16 GPUs and 8 GPUs. Figure 7 compares the baseline and PHOENIX across six topologies and four model sizes. Across all configurations, PHOENIX closely tracks the baseline. Although a few cases show small positive or negative deviations, these differences are non-systematic and are consistent with normal run-to-run and node-allocation variability, rather than a persistent overhead introduced by per-iteration checkpointing. Together with the same-topology weak-scaling results, these results show that PHOENIX introduces no observable checkpoint overhead both as system scale increases and across diverse parallel topologies.

To verify that PHOENIX’s zero-overhead property generalizes across hardware platforms, we further evaluate it with larger models on Vista’s Grace-Hopper nodes using 64 nodes. Figure 8 compares per-iteration training time for 7B, 21B, and

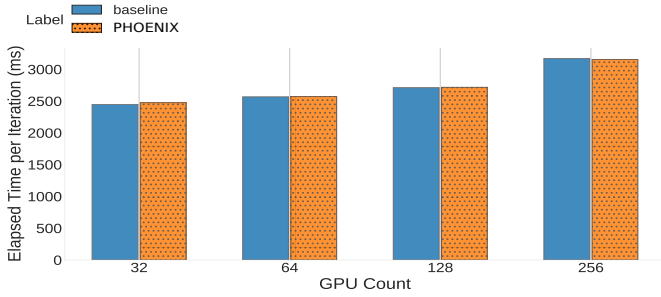


Fig. 6: Same-topology weak scaling results for the 2.3B model with tensor parallelism (TP=4) and pipeline parallelism (PP=4). The x-axis shows the number of nodes (8, 16, 32, and 64), and the y-axis reports the per-iteration training time in milliseconds. The blue bars (*baseline*) correspond to the original training configuration without PHOENIX, while the orange dotted bars (PHOENIX) show performance with PHOENIX enabled.

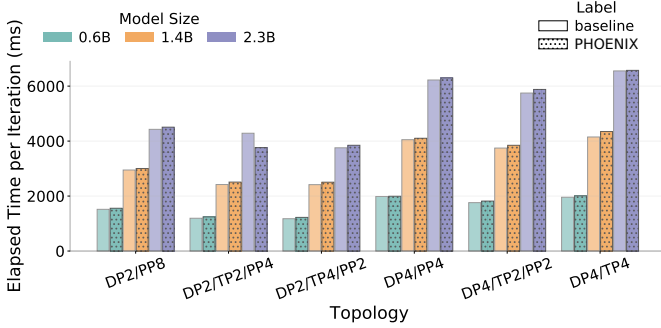


Fig. 7: Per-iteration training time across six 3D parallel topologies on 4 nodes (16 GPUs) under weak-scaling settings. The x-axis lists the training topology, and the y-axis shows elapsed time per iteration in milliseconds. Colors denote model size. For each model size within a topology, the solid bar represents the baseline configuration and the hatched bar represents PHOENIX.

65B models. Across all three scales, PHOENIX closely tracks the baseline: at 7B, PHOENIX averages 1854.5 ms versus the baseline’s 1912.8 ms; at 21B, 3311.3 ms versus 3479.4 ms; and at 65B, the two are effectively identical (7316.2 ms vs. 7312.0 ms). The small differences in either direction are system noise. These results confirm that PHOENIX’s zero-overhead property holds for models an order of magnitude larger than those tested on Perlmutter, and generalizes across GPU architectures (A100 vs. H200) and interconnect fabrics (Slingshot vs. InfiniBand).

2) *PHOENIX Communication Breakdown*: We first examine the communication breakdown under the same-topology weak-scaling setup used in the main text. Figure 9(Left) shows that as the system scales from 32 to 256 GPUs with TP=4 and PP=4 fixed, forward/backward computation remains the dominant component of iteration time, while both D2H and H2H offload costs decrease with scale. This behavior is expected because increasing the data-parallel degree reduces

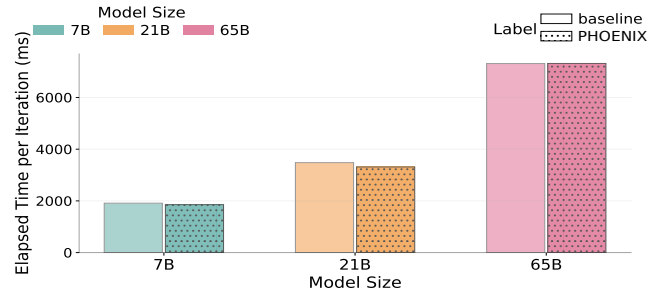


Fig. 8: Per-iteration training time for 7B, 21B, and 65B models on Vista.

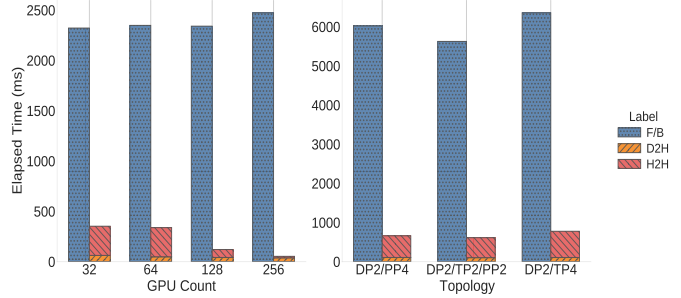


Fig. 9: Per-iteration communication breakdown of PHOENIX. (Left) Same-topology weak scaling with TP=4 and PP=4, where the x-axis shows the node count and the y-axis shows elapsed time per iteration in milliseconds. (Right) 2 nodes runs across three 3D parallel topologies, with the y-axis showing elapsed time per iteration in milliseconds. Bars are decomposed into forward/backward computation (F/B), device-to-host transfer (D2H), and host-to-host transfer (H2H).

the amount of optimizer state that must be offloaded per rank. Consequently, the offload path occupies only a small fraction of the available compute window, making it straightforward to overlap communication with ongoing computation. This explains why PHOENIX introduces negligible checkpoint overhead in weak-scaling.

We next consider a more communication-intensive setting: 2-node (8-GPU) runs across several 3D parallel topologies. In this setting, each rank carries a larger checkpoint payload than in the larger-scale weak-scaling experiments, making overlap more challenging. Figure 9(Right) shows that although D2H and H2H costs are higher in absolute terms, they still remain well below the 5.6–6.4 s forward/backward compute window across all tested topologies. Thus, even in this more demanding small-scale case, the offload path still has sufficient room to be overlapped with computation. Together, these results show that PHOENIX benefits naturally from weak scaling, but remains effective even when the per-rank offload volume is higher.

3) *Hot-swapping Recovery Latency across Scales*: We evaluate the end-to-end recovery latency of PHOENIX across both model and system scales. Our goal is to understand how hot-swapping latency depends on model size, cluster size, and memory pressure.

We first vary model sizes on a fixed 128-GPU cluster with

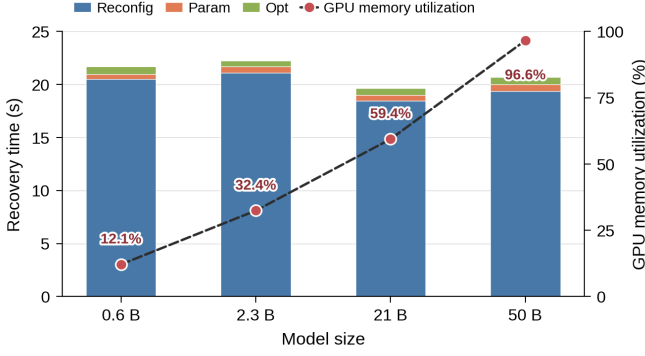


Fig. 10: Breakdown of recovery latency across model scales with measured GPU memory utilization on 128 GPUs.

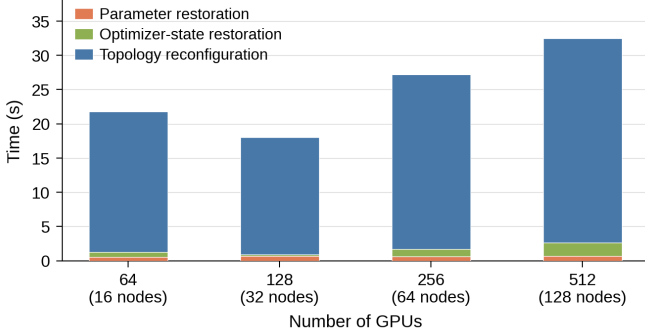


Fig. 11: Breakdown of recovery latency and end-to-end recovery time across cluster scales for GPT-7B model under fixed high per-GPU memory utilization.

a fixed parallel layout. As shown in Figure 10, recovery latency remains stable across models ranging from 0.6B to 50B parameters, with per-GPU memory utilization increasing from 12.1% to over 96.6%. This demonstrates that recovery time is independent of model scale and memory footprint.

We then scale the cluster size in a weak-scaling setup, where per-GPU workload and memory usage are held constant. As shown in Figure 11, recovery latency increases from 17.1 s to 29.8 s as the system scales from 32 to 512 GPUs. These results show that PHOENIX’s recovery latency is independent of model size and memory pressure, and grows only with system-level communication reconfiguration. This increase closely follows the growth of topology reconfiguration time, while parameter and optimizer-state restoration remain nearly constant across scales. A breakdown of the recovery path shows that communicator reconstruction (e.g., NCCL rebuild) dominates recovery latency, whereas state restoration contributes only a small and stable portion.

In comparison, restarting from checkpoint in our experiments takes on average 150 s, including distributed initialization, checkpoint loading, and process synchronization, while PHOENIX completes recovery via hot-swapping in under 40 s. This corresponds to a reduction of over $3.7\times$ in recovery latency. Notably, communicator reconstruction corresponds to a subset of the initialization steps required

in checkpoint-restart, where restarting the training job requires full `torch.distributed` initialization and additional overheads. In contrast, PHOENIX limits recovery to communication reconfiguration without full re-initialization, resulting in a more lightweight recovery path. As system scale increases and failures become more frequent, the relative advantage of PHOENIX is expected to further increase.

C. Real-World Benefit Quantification

We quantify the impact of PHOENIX under realistic failure conditions by combining measured recovery costs from a real training run with failure events generated from empirically observed failure rates. We use the failure rate reported in Meta’s large-scale training study [3] to model the occurrence of failures, and apply Monte Carlo sampling to generate failure times for our experiment.

We consider a 2.3B training job on 64 nodes for 100K steps, with persistent checkpoints taken every 5K steps (8.9 s per checkpoint), corresponding to a total runtime of 2.41 days (2.08 s per step). Based on the sampled failure process, two failure events occur at 18 and 2366 steps after the most recent checkpoint, respectively.

To resume to the original training progress, PHOENIX requires 32.6 s and 29.8 s for the two failures, respectively. In contrast, checkpoint-restart requires 150 s per recovery in addition to replaying lost work since the last checkpoint. Overall, PHOENIX reduces the failure-induced recovery time to **5.3%** of checkpoint-restart, corresponding to a **$18.8\times$** reduction in time to resume training progress. As system scale increases and failures become more frequent, the relative advantage of PHOENIX becomes more pronounced, since it avoids both repeated checkpoint loading and full-job restarts.

VIII. CONCLUSION AND FUTURE WORK

In this paper, we present PHOENIX, a fault-tolerant training system for large-scale LLMs that enables recovery via online hot-swapping rather than full job restarts. By continuously replicating optimizer-state shards in memory and reconstructing distributed communication with a spare node, PHOENIX transforms failure recovery into an online topology repair problem. We show that PHOENIX enables per-iteration checkpointing without introducing overhead on the training critical path, making fine-grained state protection practical at scale. Our evaluation further demonstrates that hot-swapping achieves efficient and scalable recovery: recovery latency grows only gradually with cluster size and is dominated by communicator reconstruction, while the cost of state restoration remains nearly constant. Together, these results indicate that PHOENIX decouples recovery performance from model size and avoids the replay and restart costs inherent in checkpoint-based approaches. This makes PHOENIX a practical and scalable solution for fault-tolerant LLM training in large-scale systems.

Future work includes improving control plane fault tolerance, extending PHOENIX to additional training frameworks, and validating its performance at larger scales.

REFERENCES

- [1] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [2] MetaSeq, “Opt-175b baselines logbook,” 2022, https://github.com/facebookresearch/metaseq/blob/main/projects/OPT/chronicles/OPT175B_Logbook.pdf.
- [3] O. Salpekar, R. Varma, K. Yu, V. Ivanov, Y. Wang, A. Sharif, M. Si, S. Xu, F. Tian, S. Zheng, T. Rice, A. Garg, S. Peng, S. Siravara, W. Fu, R. de Castro, A. Gangidi, A. Obratzov, S. Narang, S. Edunov, M. Naumov, C. Tang, and M. Oldham, “Training LLMs with fault Tolerant HSDP on 100,000 GPUs,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.00277>
- [4] D. Narayanan, M. Shoeny, J. Casper, P. LeGresley, M. Patwary, and B. Catanzaro, “Efficient large-scale language model training on gpu clusters using megatron-lm,” *arXiv preprint arXiv:2104.04473*, 2021. [Online]. Available: <https://arxiv.org/abs/2104.04473>
- [5] A. Maurya, R. Underwood, M. M. Rafique, F. Cappello, and B. Nicolae, “Datastates-llm: Lazy asynchronous checkpointing for large language models,” in *Proceedings of the 33rd international symposium on high-performance parallel and distributed computing*, 2024, pp. 227–239. doi: 10.1145/3625549.3658685.
- [6] B. Wan, M. Han, Y. Sheng, Y. Peng, H. Lin, M. Zhang, Z. Lai, M. Yu, J. Zhang, Z. Song, X. Liu, and C. Wu, “Bytecheckpoint: a unified checkpointing system for large foundation model development,” in *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation*, ser. NSDI ’25. USA: USENIX Association, 2025. [Online]. Available: <https://arxiv.org/abs/2407.20143>
- [7] B. Wu, L. Xia, Q. Li, K. Li, X. Chen, Y. Guo, T. Xiang, Y. Chen, and S. Li, “Transom: An Efficient Fault-Tolerant System for Training LLMs,” 2023. doi: 10.48550/arXiv.2310.10046. [Online]. Available: <https://arxiv.org/abs/2310.10046>
- [8] X. Lian, S. A. Jacobs, L. Kurilenko, M. Tanaka, S. Bekman, O. Ruwase, and M. Zhang, “Universal checkpointing: a flexible and efficient distributed checkpointing system for large-scale dnn training with reconfigurable parallelism,” in *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference*, ser. USENIX ATC ’25. USA: USENIX Association, 2025. doi: 10.48550/arXiv.2406.18820.
- [9] J. Mohan, A. Phanishayee, and V. Chidambaram, “CheckFreq: Frequent, Fine-Grained DNN checkpointing,” in *19th USENIX Conference on File and Storage Technologies (FAST 21)*. USENIX Association, Feb. 2021, pp. 203–216. [Online]. Available: <https://www.usenix.org/conference/fast21/presentation/mohan>
- [10] J. Thorpe, P. Zhao, J. Eyolfson, Y. Qiao, Z. Jia, M. Zhang, R. Netravali, and G. H. Xu, “Bamboo: Making preemptible instances resilient for affordable training of large DNNs,” in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 497–513. [Online]. Available: <https://arxiv.org/abs/2204.12013>
- [11] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, and L. Antiga, “Pytorch: An imperative style, high-performance deep learning library,” *Advances in neural information processing systems*, vol. 32, pp. 8026–8037, 2019. [Online]. Available: <https://arxiv.org/abs/1912.01703>
- [12] L. Bautista-Gomez, S. Tsuboi, D. Komatitsch, F. Cappello, N. Maruyama, and S. Matsuoka, “Fti: high performance fault tolerance interface for hybrid systems,” in *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’11. New York, NY, USA: Association for Computing Machinery, 2011. doi: 10.1145/2063384.2063427. [Online]. Available: <https://doi.org/10.1145/2063384.2063427>
- [13] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman *et al.*, “On the opportunities and risks of foundation models,” *Preprint arXiv:2108.07258*, 2021. [Online]. Available: <https://arxiv.org/abs/2108.07258>
- [14] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre, “Training compute-optimal large language models,” *arXiv preprint arXiv:2203.15556*, 2022. [Online]. Available: <https://arxiv.org/abs/2203.15556>
- [15] T. Ben-Nun and T. Hoefler, “Demystifying parallel and distributed deep learning: An in-depth concurrency analysis,” *ACM Computing Surveys*, vol. 52, no. 4, pp. 1–43, 2019. doi: <https://doi.org/10.1145/3320060>.
- [16] S. Li, F. Xue, C. Baranwal, Y. Li, and Y. You, “Sequence parallelism: Long sequence training from system perspective,” 2022. [Online]. Available: <https://arxiv.org/abs/2105.13120>
- [17] H. Liu and P. Abbeel, “Blockwise parallel transformers for large context models,” *Advances in Neural Information Processing Systems*, vol. 36, 2024. doi: 10.5555/3666122.3666508.
- [18] H. Liu, M. Zaharia, and P. Abbeel, “Ring attention with blockwise transformers for near-infinite context,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.01889>
- [19] B. Nie, D. Tiwari, S. Gupta, E. Smirni, and J. H. Rogers, “A large-scale study of soft-errors on gpus in the field,” in *2016 IEEE International Symposium on High Performance Computer Architecture, HPCA 2016, Barcelona, Spain, March 12-16, 2016*. IEEE Computer Society, 2016, pp. 519–530. doi: 10.1109/HPCA.2016.7446091. [Online]. Available: <https://doi.org/10.1109/HPCA.2016.7446091>
- [20] M. V. Beigi, Y. Cao, S. Gurumurthi, C. Recchia, A. Walton, and V. Sridharan, “A systematic study of ddr4 dram faults in the field,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 991–1002. doi: 10.1109/HPCA56546.2023.10071066.
- [21] V. Fratin, D. Oliveira, C. Lunardi, F. Santos, G. Rodrigues, and P. Rech, “Code-dependent and architecture-dependent reliability behaviors,” in *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2018, pp. 13–26. doi: 10.1109/DSN.2018.00015.
- [22] S. Ganapathy, J. Kalamatianos, B. M. Beckmann, S. Raasch, and L. G. Szafaryn, “Killi: Runtime fault classification to deploy low voltage caches without MBIST,” in *25th IEEE International Symposium on High Performance Computer Architecture, HPCA 2019, Washington, DC, USA, February 16-20, 2019*. IEEE, 2019, pp. 304–316. doi: 10.1109/HPCA.2019.00046. [Online]. Available: <https://doi.org/10.1109/HPCA.2019.00046>
- [23] M. Snir, R. W. Wisniewski, J. A. Abraham, S. V. Adve, S. Bagchi, P. Balaji, J. Belak, P. Bose, F. Cappello, B. Carlson, A. A. Chien, P. Coteus, N. A. DeBardeleben, P. C. Diniz, C. Engelmann, M. Erez, S. Fazzari, A. Geist, R. Gupta, F. Johnson, S. Krishnamoorthy, S. Leyffer, D. Liberty, S. Mitra, T. Munson, R. Schreiber, J. Stearley, and E. V. Hensbergen, “Addressing failures in exascale computing,” *The International Journal of High Performance Computing Applications*, vol. 28, no. 2, pp. 129–173, 2014. doi: 10.1177/1094342014522573.
- [24] D. A. G. De Oliveira, L. L. Pilla, M. Hanzich, V. Fratin, F. Fernandes, C. Lunardi, J. M. Cela, P. O. A. Navaux, L. Carro, and P. Rech, “Radiation-induced error criticality in modern hpc parallel accelerators,” in *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2017, pp. 577–588. doi: 10.1109/HPCA.2017.41.
- [25] Z. Zhang, L. Huang, R. Huang, W. Xu, and D. S. Katz, “Quantifying the impact of memory errors in deep learning,” in *2019 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 2019, pp. 1–12. doi: 10.1109/CLUSTER.2019.8890989.
- [26] Y. He, M. Hutton, S. Chan, R. De Gruijl, R. Govindaraju, N. Patil, and Y. Li, “Understanding and mitigating hardware failures in deep learning training systems,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–16. doi: 10.1145/3579371.3589105.
- [27] R. Pincirol, L. Yang, J. Alter, and E. Smirni, “Lifespan and failures of ssds and hdds: Similarities, differences, and prediction models,” *IEEE Transactions on Dependable and Secure Computing*, 2021. doi: 10.1109/TDSC.2021.3131571.
- [28] A. Sergeev and M. D. Balso, “Horovod: Fast and easy distributed deep learning in TensorFlow,” *arXiv preprint arXiv:1802.05799*, 2018. [Online]. Available: <https://arxiv.org/abs/1802.05799>
- [29] M. Jeon, S. Venkataraman, A. Phanishayee, J. Qian, W. Xiao, and F. Yang, “Analysis of large-scale multi-tenant gpu clusters for dnn training workloads,” 2019. [Online]. Available: <https://arxiv.org/abs/1901.05758>
- [30] Q. Weng, W. Xiao, Y. Yu, W. Wang, C. Wang, J. He, Y. Li, L. Zhang, W. Lin, and Y. Ding, “MLaaS in the wild: Workload analysis and scheduling in Large-Scale heterogeneous GPU clusters,” in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 2022, pp. 945–960. [Online]. Available: <https://www.usenix.org/conference/nsdi22/presentation/weng>

- [31] M. Li, W. Xiao, H. Yang, B. Sun, H. Zhao, S. Ren, Z. Luan, X. Jia, Y. Liu, Y. Li, W. Lin, and D. Qian, "Easyscale: Elastic training with consistent accuracy and improved utilization on gpus," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–14. doi: 10.1145/3581784.3607054.
- [32] Q. Wang, T. Lan, Y. Tang, Z. Huang, Y. Du, H. Zhang, J. Sha, H. Lu, Y. Zhou, K. Zhang, and M. Tang, "Dlover-rm: Resource optimization for deep recommendation models training in the cloud," 2024. [Online]. Available: <https://arxiv.org/abs/2304.01468>
- [33] S. Athlur, N. Saran, M. Sivathanu, R. Ramjee, and N. Kwatra, "Varuna: scalable, low-cost training of massive deep learning models," in *Proceedings of the Seventeenth European Conference on Computer Systems*, 2022, pp. 472–487. [Online]. Available: <https://api.semanticscholar.org/CorpusID:243847496>
- [34] I. Jang, Z. Yang, Z. Zhang, X. Jin, and M. Chowdhury, "Oobleck: Resilient distributed training of large models using pipeline templates," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 382–395. doi: 10.1145/3600006.3613152.
- [35] J. Duan, Z. Song, X. Miao, X. Xi, D. Lin, H. Xu, M. Zhang, and Z. Jia, "Parcae: Proactive, Liveput-Optimized DNN Training on Preemptible Instances," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, 2024, pp. 1121–1139. [Online]. Available: <https://arxiv.org/abs/2403.14097>
- [36] S. Gandhi, M. Zhao, A. Skiadopoulos, and C. Kozyrakis, "Slipstream: Adapting pipelines for distributed training of large dnns amid failures," *arXiv preprint arXiv:2405.14009*, 2024. doi: 10.48550/arXiv.2405.14009.
- [37] J. Mohan, A. Phanishayee, and V. Chidambaram, "CheckFreq: Frequent, Fine-Grained DNN Checkpointing," in *19th USENIX Conference on File and Storage Technologies (FAST 21)*, 2021, pp. 203–216. [Online]. Available: <https://par.nsf.gov/biblio/10286595>
- [38] Z. Wang, Z. Jia, S. Zheng, Z. Zhang, X. Fu, T. E. Ng, and Y. Wang, "Gemini: Fast failure recovery in distributed training with in-memory checkpoints," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 364–381. doi: 10.1145/3600006.3613145.
- [39] G. Wang, O. Ruwase, B. Xie, and Y. He, "FastPersist: Accelerating Model Checkpointing in Deep Learning," *arXiv preprint arXiv:2406.13768*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.13768>
- [40] T. Gupta, S. Krishnan, R. Kumar, A. Vijeev, B. Gulavani, N. Kwatra, R. Ramjee, and M. Sivathanu, "Just-in-time checkpointing: Low cost error recovery from deep learning training failures," in *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024, pp. 1110–1125. doi: 10.1145/3627703.3650085.
- [41] T. Zhang, K. Liu, J. Kosaian, J. Yang, and R. Vinayak, "Efficient fault tolerance for recommendation model training via erasure coding," *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 3137–3150, 2023. doi: 10.14778/3611479.3611514.
- [42] A. Eisenman, K. K. Matam, S. Ingram, D. Mudigere, R. Krishnamoorthi, K. Nair, M. Smelyanskiy, and M. Annavaram, "Check-N-Run: A checkpointing system for training deep learning recommendation models," in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 2022, pp. 929–943. [Online]. Available: <https://www.usenix.org/conference/nsdi22/presentation/eisenman>
- [43] K. Maeng, S. Bharuka, I. Gao, M. C. Jeffrey, V. Saraph, B.-Y. Su, C. Trippel, J. Yang, M. Rabbat, B. Lucia, and C.-J. Wu, "Cpr: Understanding and improving failure tolerant training for deep learning recommendation with partial recovery," 2020. [Online]. Available: <https://arxiv.org/abs/2011.02999>
- [44] E. Karrels, L. Huang, Y. Kan, I. Arora, Y. Wang, D. S. Katz, W. Gropp, and Z. Zhang, "Fine-grained policy-driven i/o sharing for burst buffers," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '23. New York, NY, USA: Association for Computing Machinery, 2023. doi: 10.1145/3581784.3607041. [Online]. Available: <https://doi.org/10.1145/3581784.3607041>
- [45] O. Salpekar, R. Varma, K. Yu, V. Ivanov, Y. Wang, A. Sharif, M. Si, S. Xu, F. Tian, S. Zheng, T. Rice, A. Garg, S. Peng, S. Siravara, W. Fu, R. de Castro, A. Gangidi, A. Obraztsov, S. Narang, S. Edunov, M. Naumov, C. Tang, and M. Oldham, "Training llms with fault tolerant hsdp on 100,000 gpus," 2026. [Online]. Available: <https://arxiv.org/abs/2602.00277>